

Software Engineering Lab – Software Requirement Specifications (SRS)

Team Members : 23MCCE16, 23MCCE21

Project Title : Budget Buddy

Problem Statement : A cross-platform personal finance ecosystem designed to help users manage accounts, track income and expenses, define budgets, import bank statements, and receive intelligent spending insights through analytics and machine learning.

1 Introduction

1.1 Purpose

The purpose of this project is to develop a mobile-first budget tracking application that helps users monitor financial activity, control category-wise spending, and make better money decisions. The system combines account management, transaction recording, budget tracking, goal tracking, CSV statement import, and AI-assisted insights in one application.

1.2 Scope

Budget Buddy allows authenticated users to manage personal finance data from Android, iOS, and web clients. The application provides the following major environments:

- Dashboard for total balance, account cards, weekly spending, and recent activity.
- Accounts, categories, transactions, budgets, and savings goals management.
- Insights for monthly spending analysis, category breakdowns, budget comparison, and goal progress.
- Profile and settings area for linked accounts, preferences, and bank statement import.
- Backend analytics and ML services for category prediction, behavior profiles, forecasts, alerts, and savings simulations.

1.3 Definitions, Acronyms, Abbreviations

- SRS: Software Requirement Specification.
- UI: User Interface.
- API: Application Programming Interface.
- CRUD: Create, Read, Update, Delete.
- JWT: JSON Web Token.

- ML: Machine Learning.
- UPI: Unified Payments Interface.
- CSV: Comma-Separated Values.
- INR: Indian Rupee currency code.
- REST: Representational State Transfer.

1.4 References

Expo and React Native documentation, Clerk authentication documentation, Django REST Framework documentation, PostgreSQL documentation, NativeWind documentation, Victory Native charting documentation, and the local Budget Buddy ML documentation in `bb_ML/README.md` and `bb_ML/ml_io_spec.md`.

1.5 Overall Description

Budget Buddy is a personal finance system made up of a React Native Expo frontend, a Django REST backend, a PostgreSQL database, Clerk-based authentication, and a Python ML pipeline. The application stores user-specific financial entities and provides summarized information through dashboards, charts, budget progress views, and predictive behavior services.

2 Overall Description

2.1 Product Perspective

The product is a standalone finance tracking application with a client-server architecture. The frontend uses Expo Router for navigation and communicates with a Django REST API. The backend persists data in PostgreSQL, validates users through Clerk JWT authentication, and connects to ML utilities for statement processing and category prediction.

2.2 Product Functions

The system performs the following functions:

- Register and authenticate users through email/password or Google OAuth using Clerk.
- Sync authenticated Clerk users into the backend user table.
- Create, view, update, and delete accounts, categories, transactions, budgets, and goals.

- Update account balances automatically when income or expense transactions are created, modified, or deleted.
- Display dashboard summaries including total balance, monthly spending, budget limit, account cards, weekly spending chart, and recent transactions.
- Generate monthly spending insights, category breakdowns, spike detection, and budget-aware insight cards.
- Predict transaction categories from note, merchant text, amount, hour, and weekday using a Naive Bayes category classifier when available.
- Import CSV bank statements, infer columns, create ML training rows, create transactions, and recompute behavior forecasts.
- Maintain user behavior profiles, alerts, forecast caches, recurring patterns, and savings simulations.

2.3 User Characteristics

The primary users are individuals who want to track personal spending and budgets from a smartphone or web browser. Users are expected to understand basic financial concepts such as accounts, income, expenses, categories, budgets, and savings goals. Administrators or developers may maintain backend configuration, ML artifacts, and seed category data.

2.4 User Constraints

Users must have a supported mobile device or web browser, an internet connection, and a valid authenticated account. Bank statement import depends on a readable CSV file containing identifiable date, amount, and description columns. Accurate analytics depend on consistent transaction entry, correct categories, and available backend services.

2.5 Assumptions and Dependencies

- Clerk authentication services are available and configured with valid frontend and backend environment variables.
- The backend has access to a PostgreSQL database through `DATABASE_URL`.
- The frontend is configured with `EXPO_PUBLIC_API_URL`.
- The ML classifier model may be absent; in that case category prediction should return a controlled service-unavailable response.
- Celery and Redis may be used for background behavior jobs, with synchronous fallback where implemented.
- Internet connectivity is stable enough for API communication and remote authentication.

3 Specific Requirements

3.1 Interface Requirements

3.1.1 External Interfaces

The frontend shall provide sign-in, sign-up, dashboard, account list, account create/edit, category list, category create/edit, transaction list, transaction create/edit, budget list, budget create/edit, goals, insights, profile, and CSV import screens. The backend shall expose authenticated REST endpoints under `/api/` for user sync, profile, dashboard, accounts, categories, transactions, budgets, goals, insights, ML category prediction, ML statement upload, behavior profile, forecasts, alerts, and savings simulation.

3.1.2 Hardware Interfaces

The system shall run on Android phones, iOS devices, and computers capable of running a modern web browser. Mobile devices may use secure storage, haptic feedback, document picking, and platform-specific navigation. No special hardware is required beyond normal device storage and network access.

3.1.3 Software Interfaces

The frontend shall use Expo, React Native, TypeScript, Expo Router, NativeWind, Zustand, Clerk Expo SDK, Axios or Fetch, SecureStore or local storage, and charting libraries. The backend shall use Django, Django REST Framework, PostgreSQL, PyJWT, django-cors-headers, Celery, django-celery-results, and Python ML modules. The ML pipeline shall use local CSV inputs and JSON/CSV output artifacts for model and behavior data.

3.1.4 Communication Interface

The frontend and backend shall communicate through HTTP or HTTPS REST APIs using JSON request and response bodies. Authenticated requests shall include a bearer token in the `Authorization` header. Statement upload shall use multipart form data. Cross-origin frontend access shall be controlled through CORS settings.

3.2 Functional Requirements

3.2.1 Use Case Model / Information Flow

- The user signs in with Clerk and the frontend syncs the user profile with the backend.
- The backend provisions or retrieves the user record and returns user details including onboarding status.

- The application loads accounts, categories, transactions, budgets, and dashboard summary data.
- The user records financial activity by creating income or expense transactions.
- The system updates account balances and stores transaction data in the database.
- The user configures budgets and goals to track spending limits and savings progress.
- The insights engine aggregates transactions, compares periods, detects spikes, and returns insight cards.
- The user may import a CSV bank statement; the system parses rows, predicts categories, creates transaction records, and recomputes behavior analytics.

3.2.2 Use Case Specification / Process Description

- User registers or logs into the application using email/password or Google OAuth.
- System validates the session through Clerk and synchronizes the user into the back-end database.
- User adds one or more accounts with account type and starting balance.
- User creates categories or uses global categories for organizing expenses.
- User creates a transaction by entering amount, type, account, category, date, and note.
- System validates positive amount, verifies account ownership, stores the transaction, and adjusts account balance atomically.
- User creates category budgets for the current month and tracks progress against spending.
- User creates savings goals with target amount, saved amount, and monthly contribution.
- User opens Insights to view total spending, previous-period comparison, category chart, insight cards, and goal progress.
- User imports a CSV statement from Profile; the backend identifies columns, predicts categories, imports rows, invalidates affected insight caches, and runs behavior analysis.

3.2.3 Analysis Classes / Data Dictionary

- Users: Clerk identity, email, name, phone number, profile image, currency, timestamps, and related finance records.
- Accounts: User-owned cash, bank, card, or wallet account with balance and creation date.
- Categories: Global or user-owned spending categories with icon, color, and name uniqueness per user.

- Transactions: Income or expense records linked to user, account, optional category, amount, note, date, and ML flags.
- Budgets: User, category, month, and limit records with a uniqueness constraint per user, category, and month.
- Goals: User-defined savings targets with saved amount, monthly contribution, computed progress percentage, and estimated months remaining.
- ML Training Rows: Imported transaction-like rows used for category prediction training and statement processing.
- Normalized Merchants: Raw merchant names mapped to normalized names and default categories.
- Model Predictions: Stored prediction result, confidence, model name, and metadata for a transaction.
- Recurring Patterns: Detected recurring merchants, expected amount, frequency, next due date, confidence, and active status.
- Insight Snapshots: Cached insight cards or behavioral summaries with kind, title, body, JSON data, expiration, and read state.
- User Behavior Profiles: Monthly burn rate, weekend overspend ratio, salary credit day expectation, and savings consistency score.
- Alerts and Prediction Caches: User alerts, unread state, forecast trajectory data, runout date, and model version.

3.3 Performance Requirements

Dashboard and list API requests should return within two seconds under normal network and database conditions. Transaction creation, update, and deletion should complete atomically and refresh account balance without visible inconsistency. Monthly insights should use caching where appropriate and should avoid recomputing unchanged periods on every request. Category prediction during transaction entry should respond within one second when the model is loaded. CSV imports should process records in batches and should avoid blocking unrelated users.

3.4 Logical Database Requirements

The system shall use PostgreSQL as the primary relational database. The database shall contain tables corresponding to User, Account, Category, Transaction, Budget, Goal, ML-TrainingRow, NormalizedMerchant, ModelPrediction, RecurringPattern, InsightSnapshot, UserBehaviorProfile, Alert, PredictionCache, and ModelRun. The database shall enforce unique Clerk IDs, unique user emails, unique user category names, unique user-category-month budgets, indexed transaction lookup by user, account, date, and recurring flag, and cascading deletion for user-owned records where appropriate.

3.5 Design Constraints

The frontend must use the Expo React Native stack with TypeScript and Expo Router. The UI must remain compatible with Android, iOS, and web targets. The backend must use Django REST Framework with Clerk JWT authentication and PostgreSQL. Financial write operations that affect account balance must be performed atomically. Environment variables must be used for API URL, Clerk configuration, database URL, CORS, and Celery settings. The ML system must tolerate missing model artifacts and must return controlled errors rather than crashing the API.

3.6 Software System Attributes

3.6.1 Reliability

The system should preserve financial consistency when transactions are created, updated, or deleted. Account balance updates and transaction writes should occur inside database transactions. Statement imports should skip invalid rows and report processing status.

3.6.2 Availability

The application should be available whenever the frontend, backend API, database, and Clerk services are reachable. The frontend includes mock-data fallback for API-unreachable development scenarios, but production availability depends on deployed backend and database services.

3.6.3 Security

The system shall protect user data through Clerk authentication, bearer-token API access, user-scoped database queries, and secure token storage on supported mobile platforms. Sensitive runtime values shall be stored in environment variables. Backend endpoints shall not expose records owned by other users.

3.6.4 Maintainability

The system should maintain a modular structure with separated frontend screens, reusable components, hooks, Zustand store, backend serializers, viewsets, services, tasks, and ML modules. API contracts should remain consistent across frontend hooks and backend serializers.

3.6.5 Portability

The frontend should run on Android, iOS, and web with minimal platform-specific changes. The backend should run in local or hosted Python environments with PostgreSQL connectivity. The ML pipeline should be executable separately from the API and should communicate through generated artifacts and service wrappers.

4 Supporting Information

Budget Buddy is implemented using an Expo React Native frontend, Django REST Framework backend, PostgreSQL database, Clerk authentication, and a Python-based ML bootstrap pipeline. Existing design artifacts are available under the **design/** directory, including use case, data-flow, class, sequence, and collaboration diagrams. The ML directory contains scripts and specifications for canonical ledger generation, merchant normalization, recurring-pattern detection, anomaly scoring, behavior summaries, and category classifier artifacts.